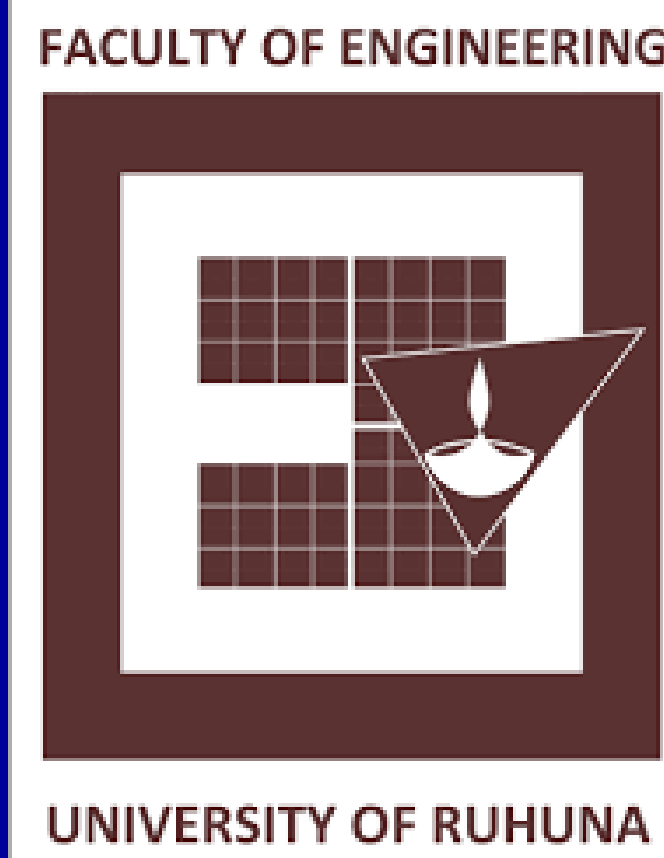




THREAT CONTAINMENT STRATEGIES FOR COMPROMISED xApps IN OPEN RAN

Dewmith M.K.J., Dilshan M.D.K., Sewinda L.L.D., Thilohith T.K.
Supervised by Dr. Chatura Seneviratne, Prof. An Braeken and Mr. Pramitha Fernando



Introduction

Open RAN rearchitects the vendor-locked traditional base station with disaggregated O-CU, O-DU and O-RU components connected by open, standardized interfaces. At the centre of it is the Near-RT RIC, a platform operating on control loops between 10 ms to 1 s, which hosts containerized applications called xApps that consume live RAN telemetry and issue control commands. These xApps subscribe to live data streams from RAN nodes via the E2 interface and run optimization logic to analyze traffic patterns, channel quality and user mobility. They send real-time commands back to RAN elements and tune parameters, manage handovers, balance network load, steer traffic and mitigate interference. Once compromised, an xApp having access to RIC platform services and a legitimate role in the control loop becomes an authorized insider on the RAN control plane.

Problem Statement

O-RAN Alliance WG11 already specifies xApp authentication, image validation, interface protection and monitoring. Currently, no reviewed system enforces runtime compromise detection to automated containment as one coordinated per-xApp architecture. Admission policies alone cannot answer three questions: Is a running xApp still inside its expected profile? What evidence justifies changing its trust state? Did enforcement actually make it isolated? Resource exhaustion is the hardest to implement, since legitimate xApps are also bursty, so a static per-metric threshold either misses the attack or fires continuously.

Objectives

The aim of this project is to implement and experimentally validate a framework that increases the resilience of the Near-RT RIC against compromised xApps. The objectives are:

1. Implement a workload identity attestation mechanism for xApps at the admission time.
2. Runtime behavioral and resource abuse threat detection for xApps.
3. Develop an automated containment pipeline triggered by security violations of xApps.
4. Experimentally validate the framework within an O-RAN testbed.

Methodology

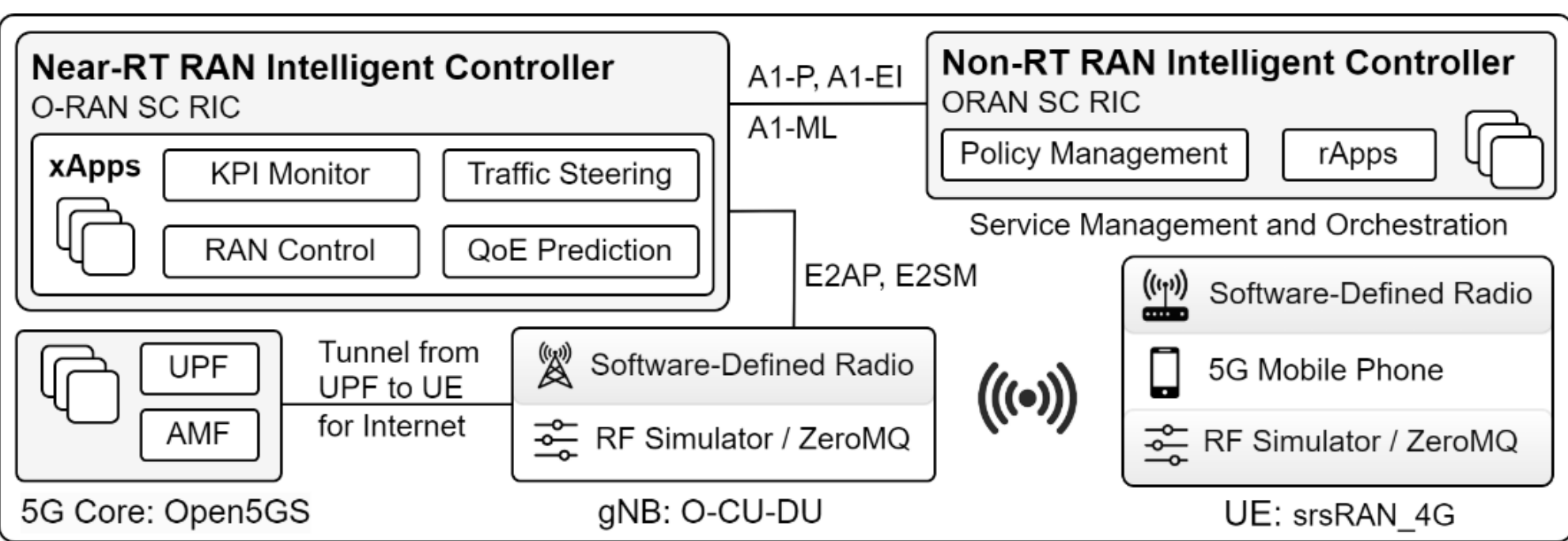


Figure 01: Testbed Architecture with 5G core, gNB, Near-RT RIC and UE

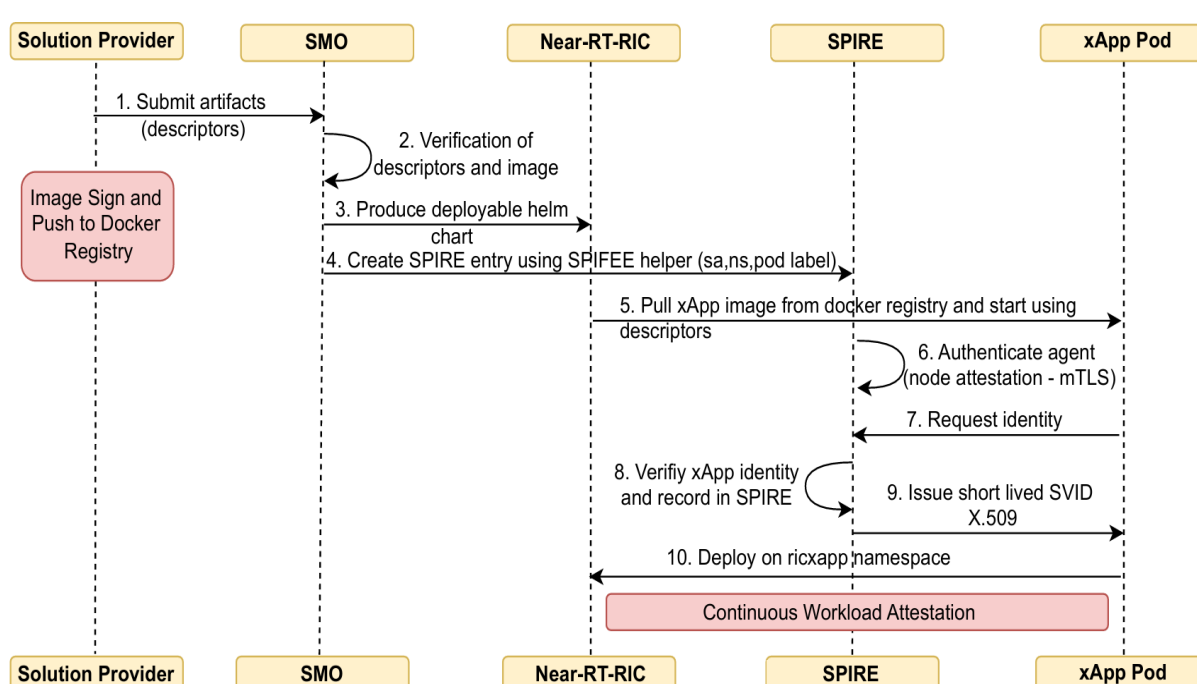


Figure 02: Workload Attestation

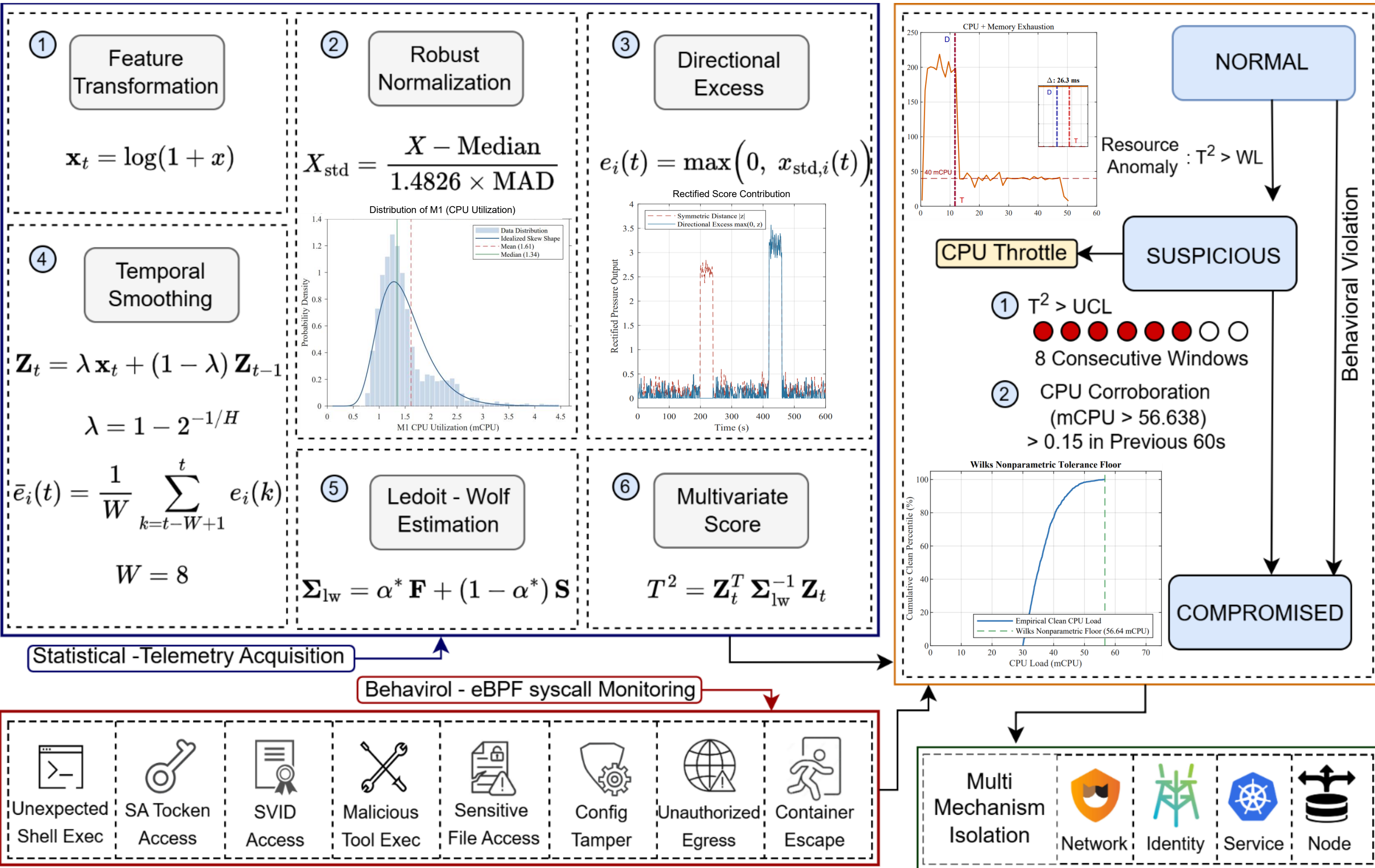


Figure 03: Overall System Architecture

Runtime Threat Detection

ZT-XGuard reassesses every running xApp through two channels with different error profiles. The behavioral channel matches kernel system calls against rules compiled from the xApp's own security profile; a CRITICAL event is high-confidence evidence and escalates immediately. The statistical channel fuses xApp resource consumption into one covariance-weighted score that must persist and be corroborated before it escalates.

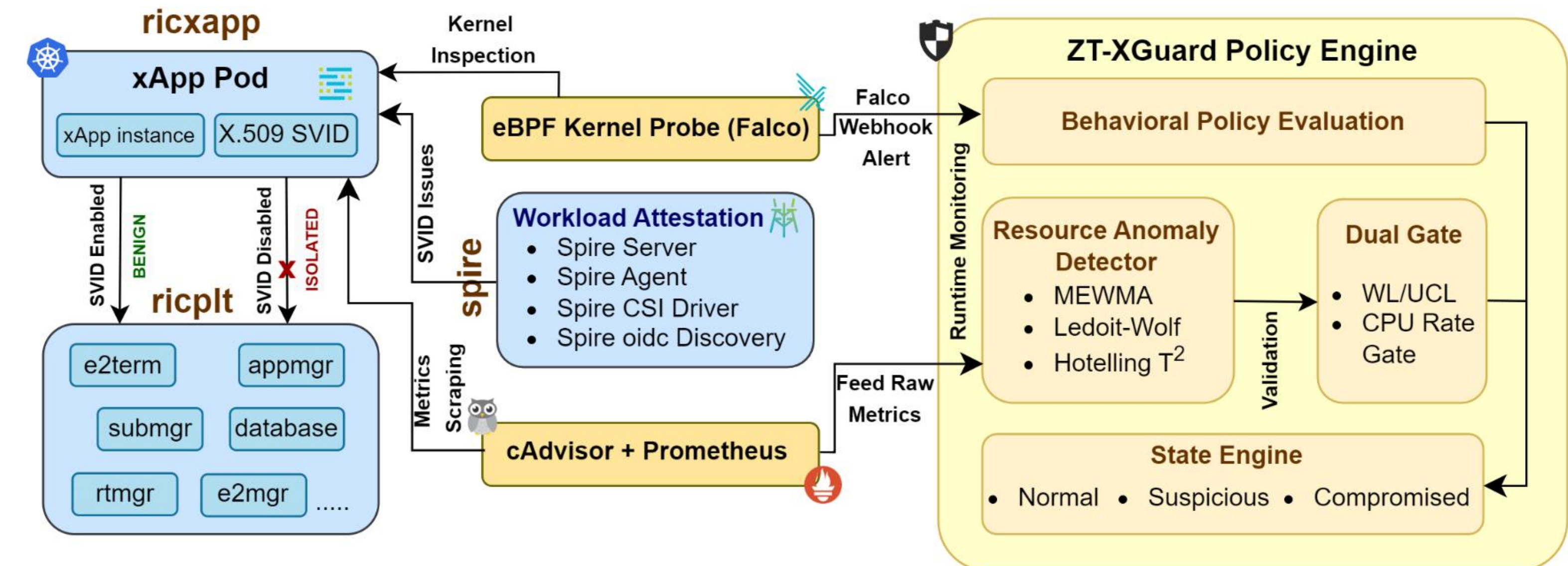


Figure 04: Runtime Threat Detection Subsystem

Trust State and Automated Containment

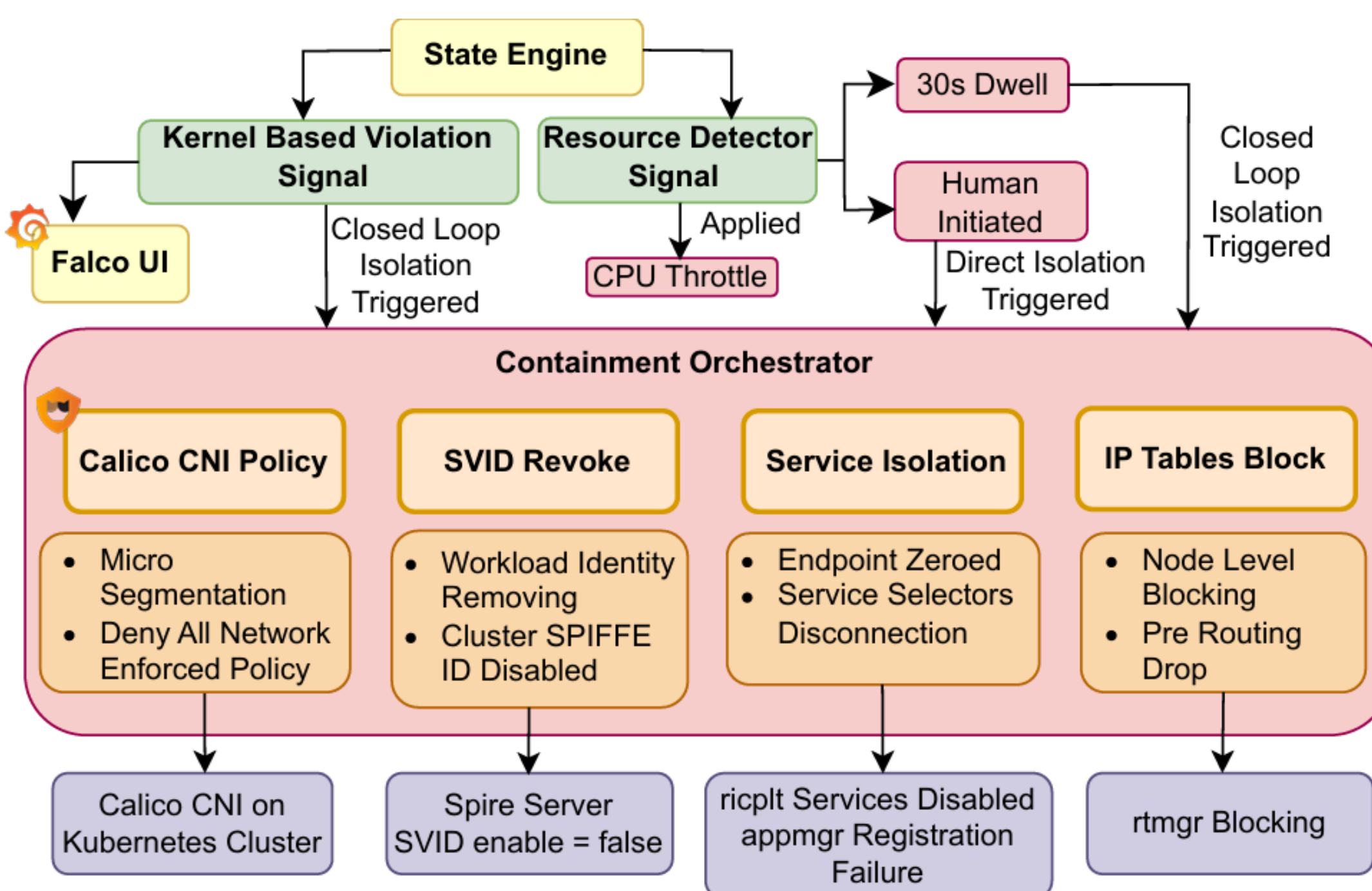


Figure 05: Automated Isolation Enforcement Subsystem

The policy engine performs decision-level fusion: each channel proposes a state independently, and proposals are serialized per xApp so a concurrent lower-severity event cannot overwrite an active higher-severity decision across the monotonic NORMAL < SUSPICIOUS < COMPROMISED states. Behavioral criticals actuate immediately, statistical confirmations hold a fixed operator-intervention dwell. Enforcement fans out concurrently across four heterogeneous layers: CNI NetworkPolicy enforcement, node-level packet filtering, Kubernetes service endpoint reconciliation and SPIFFE identity withdrawal. Then, a TCP connection through the Service path verifies the full isolation.

Performance Evaluation

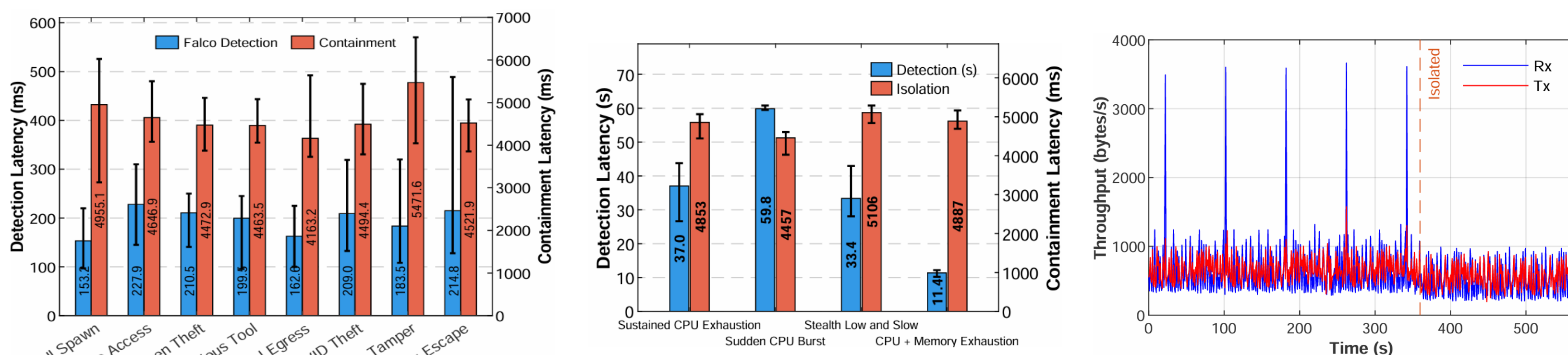


Figure 06: Behavioral Path Detection and Containment Latency

Figure 07: Statistical Path Detection and Containment Latency

Figure 08: Severance of the Isolated xApp from RIC Control Plane

Behavioral channel detection completes in 153–228 ms across all scenarios, with verified containment following in 4.16–5.47 s. Statistical channel detection spans 11.4–59.8 s depending on resource abuse pattern, while post-dwell containment stays flat at 4.46–5.11 s, confirming both channels share one enforcement path. The periodic RIC indication peaks on the xApp's E2 path cease after isolation, independently confirming removal from the live control loop.

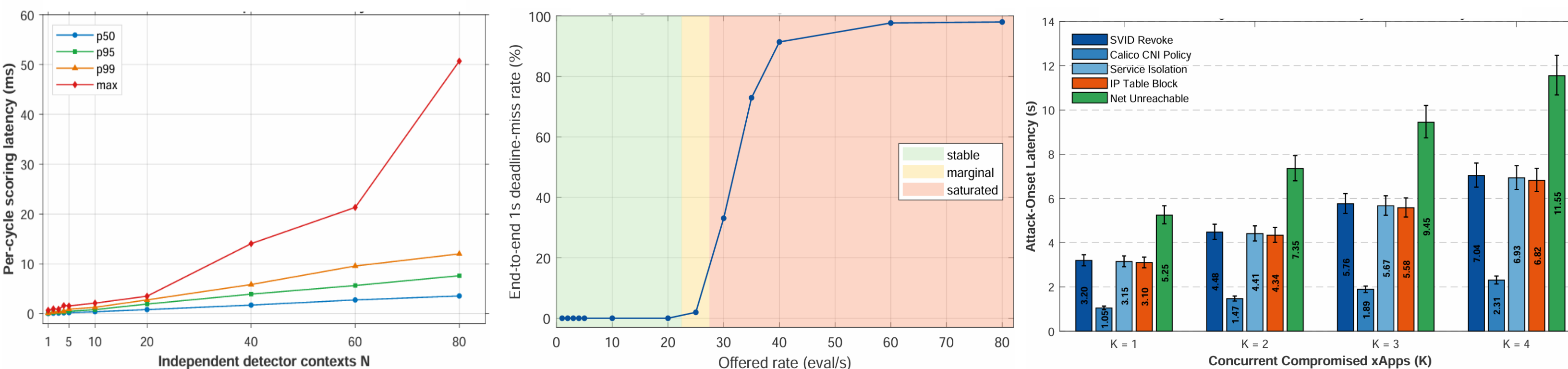


Figure 09: Detector Per-Cycle Scoring Latency

Figure 10: Policy Engine Concurrent Decision Evaluation

Figure 11: Containment Latencies for Concurrent Isolation of xApps

Detector compute per-cycle scoring latency grows near-linearly to a p99 of 12.0 ms at 80 independent detector contexts roughly 1.2% of the 1s budget with no missed cycle at any observed level. Policy decision plane path holds a zero deadline-miss rate through 20 eval/s, then saturates sharply past a knee near 25–30, making it the first component to become the binding constraint. All four mechanisms were completed and every isolated xApp was verified unreachable at each concurrency level, with verification latency growing from 5.25 s to 11.55 s across K = 1 to 4.

Conclusion

- Built and validated a zero-trust lifecycle spanning security architecture for xApps using, secure onboarding with cryptographic workload attestation, dual-channel runtime threat detection and automated containment running on a live Near-RT RIC with a real 5G core and gNB.
- Designed a multivariate resource-anomaly detector that catches sustained, duty-cycled and low and slow compute resource abuse leaving no behavioral syscall signature.
- Closed the loop from detection to isolation, verifying unreachability with an active probe instead of trusting a control-plane acknowledgement and established that evidence quality should set response timing, not a uniform policy.
- Delivered the entire response loop autonomously and offline, using statistical approaches and open-source cloud-native tools already available for a standard Near-RT RIC.

Future Work

- Generalize the statistical detector across a diverse xApp population and benchmarking against simpler detectors.
- Application-level models of correct RAN-control behaviour to reach E2 data poisoning and implement corroboration beyond a single resource dimension.
- Scale from prototype to operational control plane with multi-node placement, horizontally scaled policy ingestion, parallel enforcement across simultaneous incidents.
- Tamper-evident forensics and improve the policy engine from discrete trust states to continuous risk scored access control.